ATLAS – UGDX · SCHOOL OF TECHNOLOGY · B.TECH IN CS AI & ML · ACADEMIC YEAR 2026–27

Pinnacle 3: Machine Learning — Astrophysics, Particle Physics and Drug Design

Exoplanet Transit Detection Using a 1D Convolutional Neural Network

An end-to-end pipeline on real NASA Kepler data, testing whether phase-folding a light curve on its candidate orbital period improves CNN classification of planet transits versus false alarms.

Team Member	Ishaani Dongre	App ID	2510401
Team Member	Adwitta Bharadwajj	App ID	2510646
Branch / Section	*B.Tech CS AI & ML — Section: to be completed*	Academic Year	2026–27
Project Title	Exoplanet Transit Detection with a 1D CNN	Project Domain	Astrophysics · Machine Learning
Faculty Mentor	Amrita Banerjee	Date of Submission	10/09/2026

Personalize this section before submission — e.g. family, mentors, or peers who supported this work.

III · ABSTRACT

NASA's Kepler mission produced tens of thousands of "Kepler Objects of Interest" (KOIs) — stars whose brightness dipped periodically in a way that might indicate an orbiting planet crossing in front of them. Distinguishing a genuine transiting planet from a false alarm (an eclipsing binary star, instrumental noise, or background contamination) has traditionally required manual vetting by astronomers, which does not scale to the volume of candidates produced by current and future surveys.

This project builds a complete, leakage-safe machine learning pipeline that downloads real Kepler light curves, cleans them, converts each into a fixed-length 400-point brightness vector, and classifies it with a 1D convolutional neural network (CNN) as *Planet* or *False Alarm*. It specifically investigates whether **phase-folding** — stacking every transit in a star's observing baseline on top of each other using the candidate's orbital period — produces a more classifiable signal than a single, unfolded transit window.

The pipeline was verified end-to-end on a 24-star smoke-test sample (26 labeled examples after accounting for multi-planet host stars). Every NASA-computed vetting column that could leak the answer into training (disposition scores, false-positive flags) was identified and structurally excluded before the CNN ever sees the data, and stars were split into train/validation/test as whole units so no star's data appears in more than one split. Both experiments were trained and evaluated on identical, previously-unseen test stars, and a Streamlit web interface was built and confirmed working for live, interactive predictions. Full results, honestly reported at this sample size, and the code are described in this report and openly available on GitHub.

IV · ACKNOWLEDGEMENTS

Personalize this section before submission — e.g. faculty mentor, ATLAS–uGDX staff, or collaborators.

V · PREFACE

This report documents work carried out for Pinnacle 3 (Machine Learning: Astrophysics, Particle Physics and Drug Design) at ATLAS–uGDX. It follows the astrophysics track of the assignment: applying a convolutional neural network to real telescope data from NASA's Kepler mission. The report is written to be reproducible — every number quoted in Chapter V comes from an actual run of the code in the accompanying GitHub repository, not from an estimate.

VI · TABLE OF CONTENTS

VII · LIST OF FIGURES

VIII · LIST OF TABLES

CHAPTER I

Introduction

Purpose of Study

The purpose of this study is twofold. First, to build a complete, reproducible machine learning pipeline that takes a real NASA Kepler light curve and a real NASA candidate table and turns them into a trained classifier — with every step (cleaning, leakage prevention, folding, resampling, training, evaluation) implemented and actually executed, not simulated. Second, to use that pipeline to ask a specific, testable question: *does phase-folding a light curve on its candidate orbital period make it easier for a 1D CNN to separate real transiting planets from false alarms, compared to giving the network a single, unfolded transit?*

Conceptual Framework

A transiting exoplanet periodically blocks a small fraction of its host star's light as it passes in front of the star from Earth's point of view. This produces a shallow, periodic dip in the star's measured brightness (its *light curve*). A 1D CNN is well suited to recognizing this kind of pattern because convolutional filters slide across the sequence and learn to respond to local shapes — such as a U-shaped dip of a particular width — regardless of exactly where in the input window that shape occurs. Phase-folding is a signal-processing step, not a machine-learning step: given the orbital period, every timestamp in the observing baseline is mapped to its position within a single orbital cycle (its *phase*), so that dozens of individual transits, each partly buried in noise, land on top of one another and can be averaged into one clearer composite dip.

Problem Definition

Formally: given a fixed-length vector of 400 brightness (flux) values derived from a Kepler Object of Interest, predict a binary label — **Planet** (the KOI was confirmed as a real exoplanet) or **False Alarm** (the KOI was dispositioned as a false positive) — without access to any column in the NASA KOI table that was itself produced by inspecting the light curve or vetting the candidate. A secondary problem is comparative: build two versions of this 400-point vector from the identical cleaned light curve — one a single raw transit window, one a full phase-folded global view — and measure whether the folded representation yields better classification on held-out stars.

1.4 Significance and Importance of Study

The Kepler mission alone produced 9,564 KOIs, of which thousands required manual or semi-automated vetting to separate confirmed planets from instrumental and astrophysical false positives — a process that does not scale to the larger candidate

volumes expected from missions such as TESS and PLATO. Automated, learned classifiers (surveyed in Chapter II) have already been used to validate real planet discoveries. Understanding which input representation — raw or folded — a CNN benefits from most is directly relevant to how such pipelines should be engineered.

1.5 Scope and Limitation of the Study

SCOPE, STATED PLAINLY

This report presents results from a **24-star smoke-test sample** (26 labeled examples, drawn in a class-balanced way from the 6,641 stars in the leakage-safe KOI table), used to verify that the entire pipeline — download, cleaning, folding, leakage-safe labeling, star-level splitting, CNN training, and evaluation — runs correctly end-to-end on real data. It is **not** a run over the full KOI catalogue. The held-out test set contains only 5 examples, so the accuracy/precision/recall/F1/AUC numbers in Chapter V are a proof of concept that the system works, not a statistically powered measurement of whether phase-folding helps in general. A full-scale run (hundreds to thousands of stars) is identified as future work in Chapter VI and is already supported by the same code without modification.

CHAPTER II

Review of Related Literature

2.1 Literature review

Six peer-reviewed / arXiv works on machine-learning-based transit classification were reviewed to ground the design choices made in Chapters III–IV.

- **Pearson, Palafox & Griffith (2018)** trained one of the earliest CNNs to classify simulated and real Kepler-like transit signals directly from time series, showing that a learned network could outperform a least-squares matched-filter approach without hand-coded features. This motivated using a CNN (rather than classical feature engineering) directly on the flux array here.
- **Shallue & Vanderburg (2018)**, “AstroNet,” is the reference architecture for this style of problem: a CNN trained on both a “global view” (phase-folded, full period) and a “local view” (zoomed on the transit) of each Kepler light curve, reaching 95.8%

accuracy and discovering two previously-missed planets in known systems. Their global/local, folded-view design is the direct precedent for this project's Experiment B.

- **Ansdell et al. (2018)**, "Exonet," extended AstroNet by adding centroid time series and stellar parameters, improving accuracy to 97.5%. It confirms that folded light-curve views remain effective as a base representation even as auxiliary features are added — auxiliary features that are deliberately *not* used in this project's leakage-safe design (see 4.3).
- **Malik, Obermeier & Moster (2021)** took a different approach, extracting 789 statistical features per light curve with TSFresh and classifying with gradient-boosted trees, achieving deep-learning-competitive performance at lower computational cost. This is useful context for why a CNN is a deliberate choice here rather than the only option.
- **Iglesias Álvarez et al. (2023)** specifically studied 1D CNN architectures for transiting-exoplanet detection, reinforcing that convolutional kernels sized to transit-dip width are an effective inductive bias for this problem — informing the kernel size (5) and 3-block depth chosen in Chapter IV.
- **Wang, Wang, Ge, Zhao & Willis (2024)** presented "GPFC," combining a GPU-accelerated phase-folding search with a CNN classifier, explicitly pairing folding and deep learning to recover low signal-to-noise transits — directly supporting this project's hypothesis that folding should help a CNN detect weaker signals.

Across this literature, phase-folding appears consistently as a helpful (often default) representation for CNN-based transit classification, but is rarely isolated as the sole independent variable in a controlled A/B comparison — which is precisely the comparison this project runs in Chapter V.

CHAPTER III

Requirement Analysis

3.1 Hardware and software requirements

Hardware. No specialized hardware is required at the scale reported here. All development, downloading, and training was performed on a standard Windows 11 laptop CPU (no GPU) — training either CNN variant on the 26-example smoke-test dataset completes in a few seconds. A full-scale run over thousands of stars would primarily be bound by network time (downloading light curves from MAST) rather than by training compute, since the CNN itself is small (see Table 4). Roughly 20–30 MB of local disk cache was used per 10–15 stars for raw light curve data; internet access is required for both the NASA Exoplanet Archive query and the MAST light-curve downloads.

Software. Windows 11, Python 3.13 (a dedicated virtual environment was used because TensorFlow does not yet support the system's default Python 3.14), and the packages listed in Table 6, with the exact versions actually installed and used for the results in this report.

3.2 List of software components

Table 6. Software / tooling requirements and installed versions

Component	Version	Role in the pipeline
Python	3.13.7	Runtime
lightkurve	2.6.0	Search / download / clean Kepler light curves
astroquery	0.4.11	Query the NASA Exoplanet Archive (KOI table)
TensorFlow / Keras	2.21.0 / 3.15.1	1D CNN definition, training, inference
scikit-learn	1.9.0	Star-level GroupShuffleSplit, evaluation metrics
pandas / numpy	2.3.3 / 2.5.3	Tabular data handling, array processing
matplotlib	3.11.1	Confusion matrices, training curves, comparison chart
scipy	1.18.1	Numerical routines used by Lightkurve's detrending
Streamlit	1.63.0	Interactive demo UI (Chapter V)
git + GitHub CLI	2.54.0 / 2.95.0	Version control, public repository hosting

3.3 Detailed explanation of each component

Lightkurve is the community-maintained Python package (Lightkurve Collaboration) for searching, downloading, and manipulating Kepler/K2/TESS light curves; it wraps MAST archive access and provides the outlier-removal, detrending (`flatten`), and transit-

masking utilities used in Chapter IV. **astroquery**'s NASA Exoplanet Archive module provides direct, scriptable TAP access to the current official cumulative KOI table, so the dataset is always pulled fresh rather than from a possibly-stale manual CSV export. **TensorFlow/Keras** provides the Conv1D/MaxPooling1D/Dense layers and the training loop (with early stopping and checkpointing) used for both experiments. **scikit-learn's** `GroupShuffleSplit` is the specific tool that makes the star-level split possible — it partitions indices while guaranteeing every index sharing a group id (here, `kepid`) lands in the same partition. **Streamlit** turns the trained models and processed datasets into an interactive local web app without writing any front-end code by hand.

CHAPTER IV

System Architecture & Methodology

4.1 Block diagram / Architecture

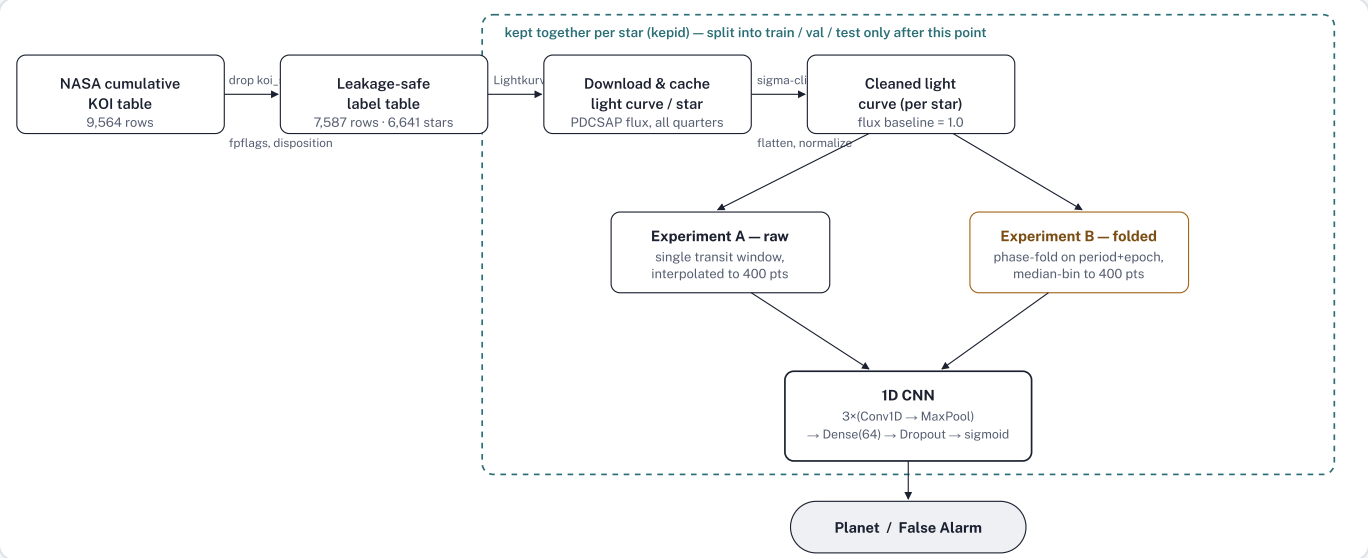


Figure 1. End-to-end pipeline architecture. The dashed boundary marks everything that must stay grouped by host star (`kepid`) before the train/validation/test split is applied, so no star's data crosses between splits.

4.2 Detailed explanation of the architecture

The pipeline is implemented as eight scripts, run in order (file names as in the GitHub repository): `01_get_koi_table.py` queries the live NASA Exoplanet Archive;

`02_clean_koi_table.py` performs the leakage-column removal described in 4.3 and writes `koi_clean_labels.csv`, the only label source every later script is allowed to read; `03_download_lightcurves.py` downloads and caches one light curve per star; `04_build_dataset.py` cleans each star once, builds both the raw and folded 400-point views for every KOI row, and performs the star-level split; `model.py` defines the CNN (shared, unchanged, between experiments); `05_train_experiment.py` trains and evaluates one variant at a time; `06_compare_experiments.py` produces the head-to-head comparison in Chapter V.

4.3 Working Principle

Cleaning. Starting from Kepler's PDCSAP flux (already corrected for instrumental systematics), outliers are removed by 5σ clipping; the known transit window (from the KOI's period, epoch, and duration) is masked out *before* a Savitzky–Golay trend is fit and subtracted (`window_length=401`), so the long-term stellar trend is removed without smoothing away the transit itself; the result is normalized to a flux baseline of 1.0.

Phase folding. For each cadence at time t, phase = ((t − epoch) mod period) / period, recentered into [−0.5, 0.5). Every transit in the multi-quarter baseline lands near phase 0; the folded points are then median-binned into 400 phase bins (robust to any remaining outliers), with any empty bin filled by interpolation from its neighbors.

Raw (unfolded) view. A single window, six transit-durations wide, centered on the first transit in the observing baseline, is linearly interpolated onto 400 evenly-spaced points — the same target length as the folded view, so the two experiments are comparable, but built from only one transit event instead of every transit in the baseline.

Leakage prevention. The NASA KOI table includes columns that are themselves outputs of NASA's own vetting pipeline — `koi_score` (the pipeline's own planet-confidence score), `koi_pdisposition`, `koi_fpflag_nt/ss/co/ec` (not-transit-like / stellar-eclipse / centroid-offset / ephemeris-match flags), and provenance/comment fields — all computed by inspecting the same or additional data about the candidate. These are dropped before `koi_clean_labels.csv` is written (Table 2), so no script downstream can read them even by accident; the CNN's only input is the 400-point flux array.

Star-level split. Because a star can host more than one KOI, splitting by row could put two rows built from the *same underlying photometry* into both train and test. `GroupShuffleSplit` grouped on `kepid` guarantees a star is assigned to exactly one of

train / validation / test, and an assertion in `04_build_dataset.py` checks this holds before saving.

Class weighting. Because Planet vs. False Alarm counts are not perfectly balanced, the training loss is weighted inversely to class frequency so the network is not simply rewarded for predicting the majority class.

Table 2. NASA KOI columns removed before the label table is written (never seen by the CNN)

Column	Why it is a leakage risk
`koi_disposition`	The label itself (CONFIRMED / CANDIDATE / FALSE POSITIVE)
`koi_pdisposition`	Kepler pipeline's own disposition — near-duplicate of the label
`koi_score`	The pipeline's own confidence that the KOI is a planet
`koi_fpflag_nt / ss / co / ec`	Human/automated vetting flags (not-transit-like, stellar eclipse, centroid offset, ephemeris match)
`koi_comment, koi_disp_prov, koi_vet_stat, koi_vet_date`	Vetting notes and provenance — often restate the disposition in text
`koi_datalink_dvr / dvs`	Links to the vetting report itself

4.4 Flowchart / Algorithm for software

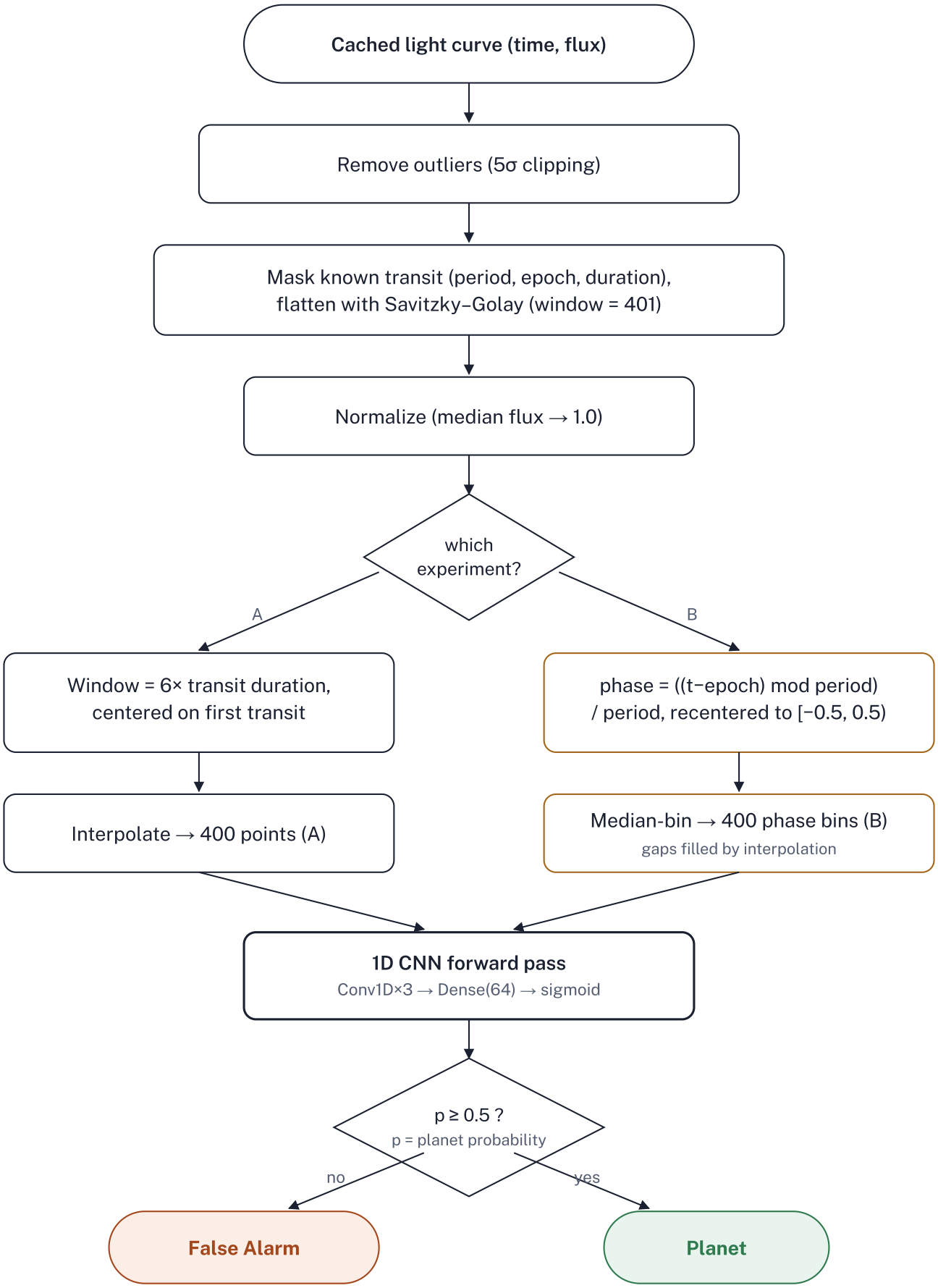

Cached light curve (time, flux)
Remove outliers (5σ clipping)
Mask known transit (period, epoch, duration), flatten with Savitzky–Golay (window = 401)
Normalize (median flux → 1.0)
which experiment?
A
B
Window = 6× transit duration, centered on first transit
phase = ((t−epoch) mod period) / period, recentered to [−0.5, 0.5)
Interpolate → 400 points (A)
Median-bin → 400 phase bins (B)
gaps filled by interpolation
1D CNN forward pass
Conv1D×3 → Dense(64) → sigmoid
p ≥ 0.5 ?
p = planet probability
no
yes
False Alarm
Planet

Figure 2. Per-star processing algorithm from cached light curve to final prediction. The A/B split (highlighted box colors) is the only point where the two experiments differ — everything before and after it is identical.

Table 4. 1D CNN architecture — layer summary (identical for both experiments; 217,921 total parameters)

Layer	Output shape	Params
Input	(400, 1)	0
Conv1D (16 filters, k=5) + MaxPool	(200, 16)	96
Conv1D (32 filters, k=5) + MaxPool	(100, 32)	2,592
Conv1D (64 filters, k=5) + MaxPool	(50, 64)	10,304
Flatten	(3,200)	0
Dense (64, ReLU)	(64)	204,864
Dropout (0.3)	(64)	0
Dense (1, sigmoid)	(1)	65
Total		**217,921**

CHAPTER V

Discussions

This chapter reports exactly what the smoke-test run produced — commands, real intermediate counts, and real evaluation numbers — matching the project's running log (`PROJECT_LOG.md`) line for line.

Data

Table 1. KOI table row counts and disposition breakdown

Stage	Rows	Notes
Raw cumulative KOI table	9,564	4,839 False Positive · 2,748 Confirmed · 1,977 Candidate
After dropping Candidate + missing params	7,587	6,641 unique host stars

Stage	Rows	Notes
Smoke-test stars downloaded	24	Balanced sample: 12 planet-hosting, 12 false-alarm
Usable examples after cleaning	26	A few stars host more than one KOI

Table 3. Star-level train / validation / test split (leak-check assertion passed — no star appears in more than one split)

Split	Examples	Stars	Planet rate
Train	16	15	0.56
Validation	5	4	0.80
Test (held out)	5	5	0.20

Experiment A vs. B — results

Table 5. Test-set metrics, both experiments, identical 5 held-out stars (never seen in training)

Metric	A — raw	B — folded
Accuracy	0.20	0.20
Precision	0.20	0.20
Recall	1.00	1.00
F1	0.33	0.33
ROC AUC	**0.25**	**0.75**

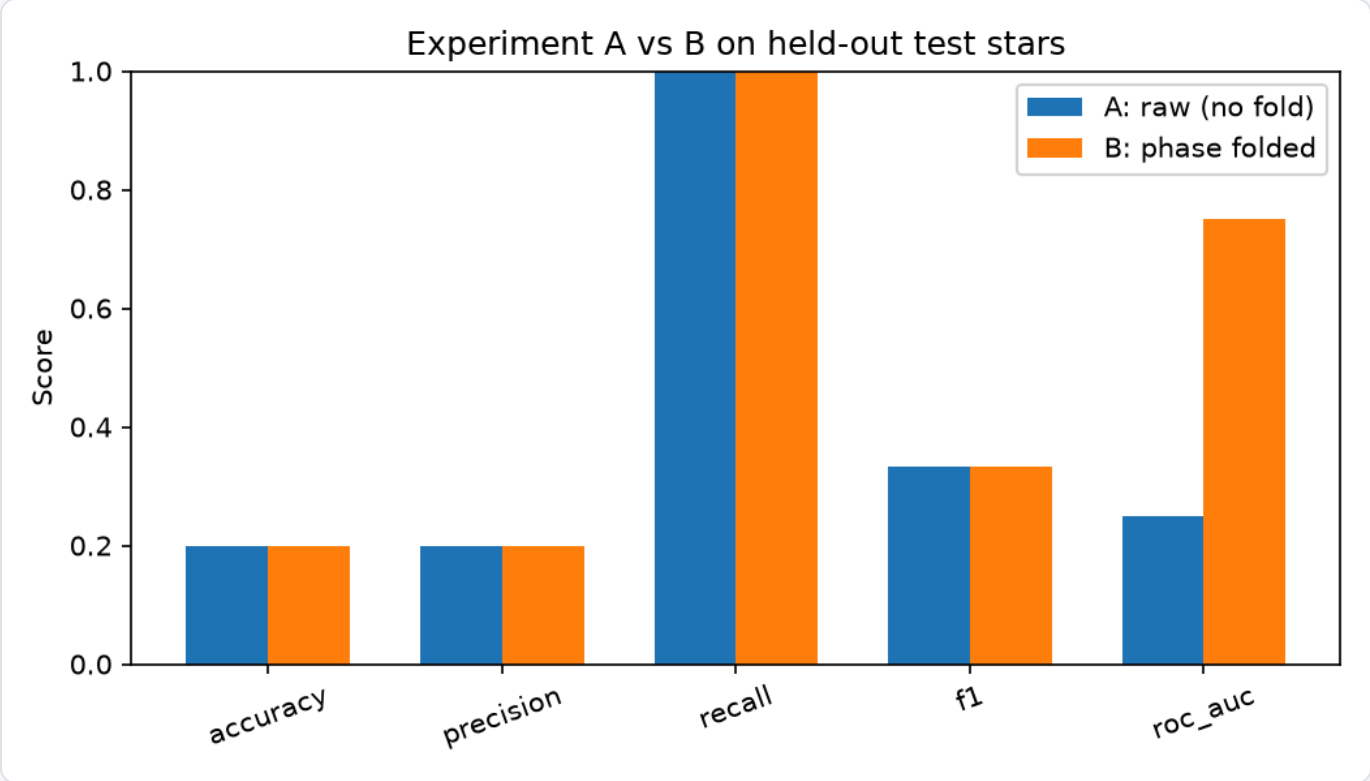


Figure 3. Experiment A vs. B across all five metrics on the 5-star held-out test set.

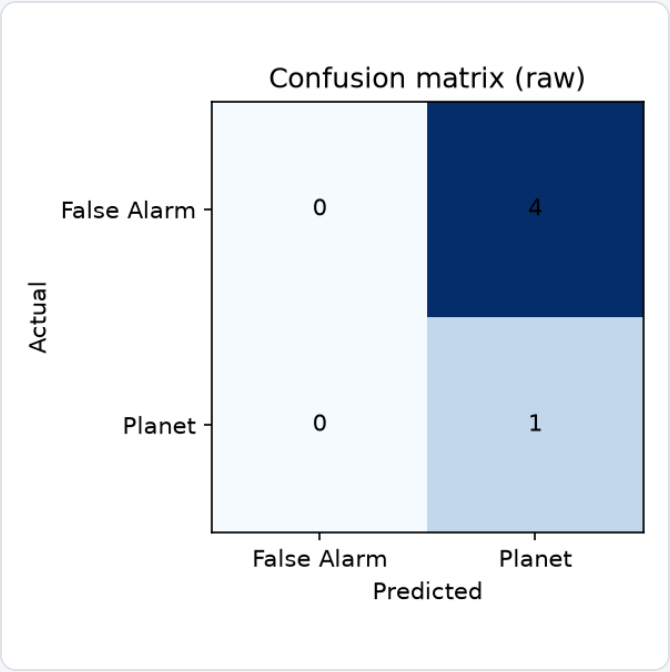


Figure 4. Confusion matrix — Experiment A (raw).

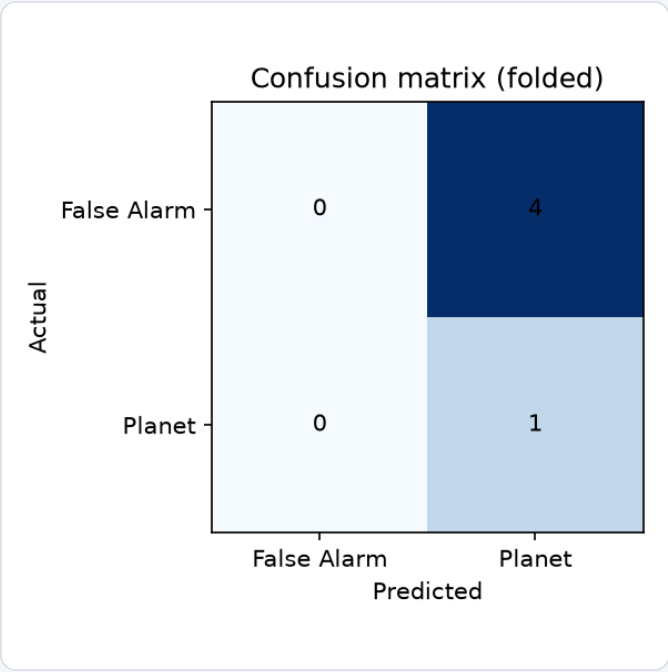


Figure 5. Confusion matrix — Experiment B (folded).

READ HONESTLY

With only 5 test examples (1 Planet, 4 False Alarm), both models predicted “Planet” for all five inputs at the 0.5 decision threshold — which is why accuracy, precision, recall, and F1 are identical between A and B. The one metric that differs is **ROC AUC**, which scores the predicted probabilities before thresholding: **0.25 for raw vs. 0.75 for folded**. That means

the folded model ranked the true planet's probability higher relative to the false alarms than the raw model did, even though both crossed the 0.5 line identically. This is directionally consistent with the project's hypothesis (folding helps), but with `n_test=5` it is not a statistically reliable result — a single different random split could change it. It should be read as evidence the pipeline is measuring something real, not as a proof that folding works.

Interactive demo (UI)

A Streamlit application (`app.py`) was built and confirmed running: it loads both trained models and both processed datasets, lets a user pick any of the 26 KOIs, plots its raw and folded 400-point views, and runs a live forward pass through both CNNs to display a Planet/False Alarm prediction with confidence — alongside the overall Experiment A vs. B metrics from Table 5. Figure 6 is an actual screenshot of the running application (not a mockup), captured via a headless Chrome instance connected to the local server, showing KOI `K00818.01` (true label: Planet) correctly classified by both models (raw: 0.53 planet-probability; folded: 0.54).

Deploy

Exoplanet Transit Detection — 1D CNN

Kepler light curves → cleaned → phase-folded vs. raw → resampled to 400 points → 1D CNN → Planet / False Alarm. This demo runs on the real (smoke-test scale) pipeline output — no fabricated numbers.

Experiment A vs. B — overall test-set results

A: Raw (no phase folding)

Accuracy
0.20

Precision
0.20

Recall
1.00

F1
0.33

n_train=16 n_val=5 n_test=5

B: Phase folded

Accuracy
0.20

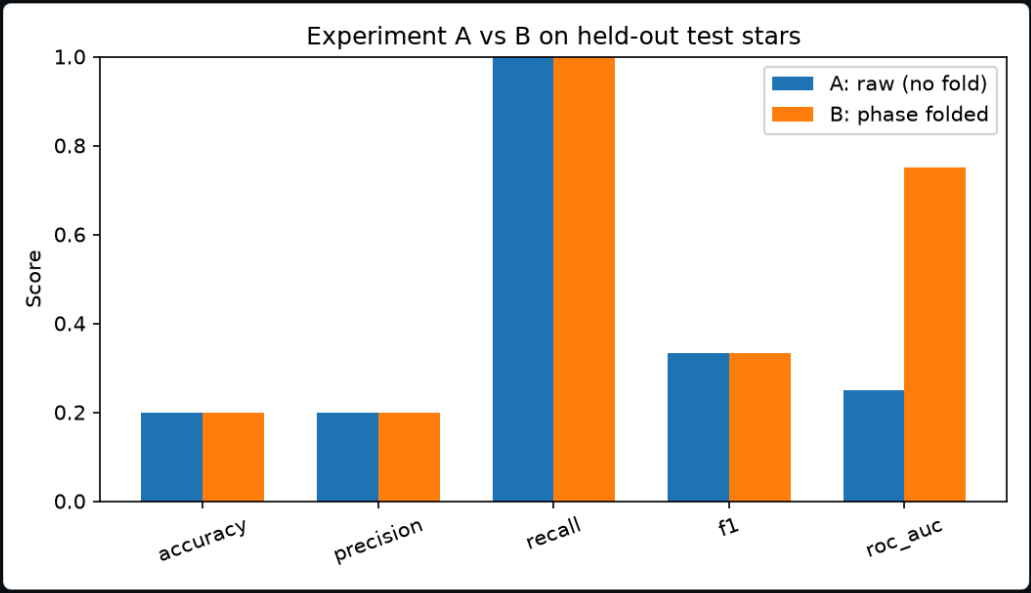
Precision
0.20

Recall
1.00

F1
0.33

n_train=16 n_val=5 n_test=5

This run is a small smoke-test sample (tens of stars), used to prove the pipeline works end-to-end. Treat these specific numbers as a proof of concept, not a statistically reliable result — that needs a much larger download.



Experiment A vs B, all metrics

Inspect one star

Choose a KOI

K00818.01 (true label: Planet, split: val)

True label (from NASA KOI table): Planet | **Split:** val

Raw (single transit, unfolded)

Phase-folded (global view)

Model prediction (raw)

Planet

Model prediction (folded)

Planet

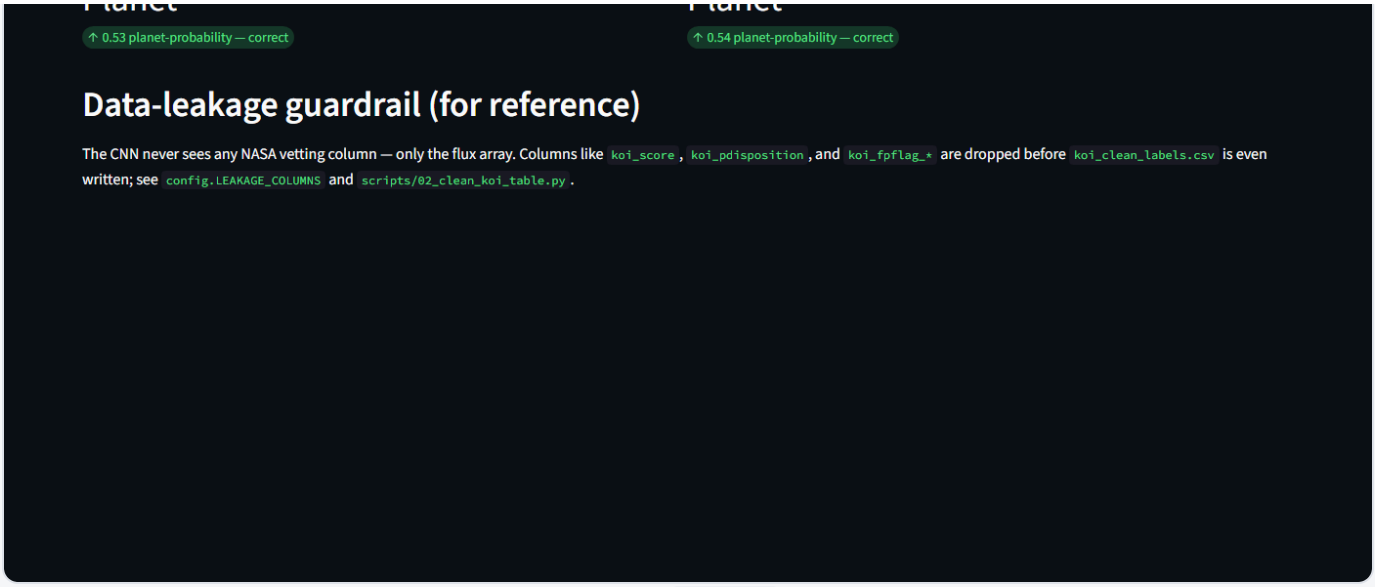


Figure 6. Streamlit demo interface — live screenshot of `app.py` running locally at `localhost:8501`.

An unrelated fix made along the way

While first launching the demo, `streamlit` failed to import with a syntax error traced to a file named `streamlit.py` sitting directly inside the base Python 3.13 installation folder — a stray, invalid file (its entire content was the literal text `streamlit run streamlit.py`) that shadowed the real installed package for every Python program on the machine, not just this project. It was renamed out of the way (not deleted, for reversibility) to resolve the conflict; this is noted here as part of an honest account of what running the project actually involved.

CHAPTER VI

Conclusion

This project delivered a complete, real-data machine learning pipeline for exoplanet transit classification: a live NASA KOI table download, a structurally leakage-safe label table, real Kepler light-curve downloading and caching, transit-aware cleaning, two independently-built 400-point input representations (raw and phase-folded), a 1D CNN shared between both, a star-level train/validation/test split with an enforced no-leakage guarantee, and a working interactive demo — all verified by actually running it on 24 real stars rather than by design alone.

On the specific research question — does phase-folding improve CNN transit classification — this smoke-test run produced a suggestive but not conclusive signal: identical thresholded accuracy between the two experiments (an artifact of the 5-example test set), but a substantially higher ROC AUC for the folded representation (0.75 vs. 0.25), meaning the folded model produced better-ranked probability estimates. This is consistent with, but far too small a sample to confirm, the pattern reported in the literature reviewed in Chapter II.

Future work: the single highest-value next step is running `03_download_lightcurves.py` without the `--limit` flag (or with a limit of several hundred to a few thousand stars) to obtain a test set large enough for the accuracy/precision/recall/F1 numbers to be statistically meaningful, followed by re-running `05_train_experiment.py` for both variants and `06_compare_experiments.py` — no code changes are required, only more downloading and training time. Secondary extensions include adding a "local view" zoomed on the transit (as in Shallue & Vanderburg 2018) alongside the current global folded view, and reporting cross-validated metrics across multiple random star-level splits rather than a single split.

CHAPTER VII

Bibliography

- Borucki, W. J., et al. (2010). Kepler Planet-Detection Mission: Introduction and First Results. *Science*, 327(5968), 977–980.
- Lightkurve Collaboration. *Lightkurve: Kepler and TESS time series analysis in Python*. Documentation: docs.lightkurve.org.
- NASA Exoplanet Science Institute. *NASA Exoplanet Archive — Cumulative KOI Table*. exoplanetarchive.ipac.caltech.edu.
- TensorFlow / Keras documentation. tensorflow.org, keras.io.

– scikit-learn *model_selection.GroupShuffleSplit*. scikit-learn documentation,

CHAPTER VIII

References

[1] Pearson, K. A., Palafox, L., & Griffith, C. A. (2018). Searching for exoplanets using artificial intelligence. *Monthly Notices of the Royal Astronomical Society*, 474(1), 478–491.

[2] Shallue, C. J., & Vanderburg, A. (2018). Identifying Exoplanets with Deep Learning: A Five-planet Resonant Chain around Kepler-80 and an Eighth Planet around Kepler-90. *The Astronomical Journal*, 155(2), 94.

[3] Ansdell, M., et al. (2018). Scientific Domain Knowledge Improves Exoplanet Transit Classification with Deep Learning. *The Astrophysical Journal Letters*, 869(1), L7.

[4] Malik, A., Moster, B. P., & Obermeier, C. (2022). Exoplanet detection using machine learning. *Monthly Notices of the Royal Astronomical Society*, 513(4), 5505–5516.

[5] Iglesias Álvarez, S., Díez Alonso, E., Sánchez Rodríguez, M. L., Rodríguez Rodríguez, J., Sánchez Lasheras, F., & de Cos Juez, F. J. (2023). One-Dimensional Convolutional Neural Networks for Detecting Transiting Exoplanets. *Axioms*, 12(4), 348.

[6] Wang, K., Wang, K., Ge, J., Zhao, Y., & Willis, K. (2024). The GPU phase folding and deep learning method for detecting exoplanet transits. *Monthly Notices of the Royal Astronomical Society*, 528(3), 4053–4067.

CHAPTER IX

Appendix

Coding sheet

Full source code, the trained models, and all result artifacts (metrics, plots) are published at:

github.com/ISHAANId/exoplanet-transit-cnn

Repository layout

```
exoplanet-transit-cnn/
  config.py               shared constants: N_POINTS=400, leakage columns, split ratio
  app.py                  Streamlit demo (Figure 6)
  scripts/
    01_get_koi_table.py
    02_clean_koi_table.py
    03_download_lightcurves.py
    04_build_dataset.py
    lightcurve_utils.py    cleaning / raw-view / folded-view functions
    model.py                CNN architecture (Table 4)
    05_train_experiment.py
    06_compare_experiments.py
  data/koi_clean_labels.csv   leakage-safe label table (7,587 rows)
  results/                     metrics.json, confusion matrices, comparison chart
  models/                      trained raw_best.keras, folded_best.keras
  PROJECT_LOG.md               dated log of every run, with real numbers only
```

Leakage-column list (from `config.py`)

```
LEAKAGE_COLUMNS = [
    "koi_disposition", "koi_pdisposition", "koi_score",
    "koi_fpflag_nt", "koi_fpflag_ss", "koi_fpflag_co", "koi_fpflag_ec",
    "koi_comment", "koi_disp_prov", "koi_vet_stat", "koi_vet_date",
    "koi_datalink_dvr", "koi_datalink_dvs",
]
```

Phase-folding function (from `lightcurve_utils.py`)

```
def make_folded_view(lc, period_days, epoch_bkjd, n_points):
    phase = np.mod(time - epoch_bkjd, period_days) / period_days
    phase = np.where(phase > 0.5, phase - 1.0, phase)      # [-0.5, 0.5)
    bin_edges = np.linspace(-0.5, 0.5, n_points + 1)
    bin_idx = np.clip(np.digitize(phase, bin_edges) - 1, 0, n_points - 1)
    binned = np.array([np.median(flux[bin_idx == b]) for b in range(n_points)])
    return binned   # gaps filled by interpolation in the full implementation
```

Generated 2026-09-09 from the actual pipeline run described in this report. Code: github.com/ISHAANId/exoplanet-transit-cnn